

Coding Project 1: Linear Regression and Regularization

Project overview, modelling workflow, and reusable ML patterns

Zhexin Wu

September 23, 2026

What should you get from today's session?

- Understand the **structure of Project 1**: what each question is trying to achieve and how the pieces connect.
- Recognize the different roles of **scikit-learn** and **statsmodels**.
- Learn the reusable scikit-learn pattern:

`preprocessing → model → fit → predict.`

- Understand why **Pipeline**, cross-validation, and hyperparameter tuning are useful abstractions.
- Know what evidence and outputs your notebook and final report should contain.

Project 1 in one picture

Q1

Prepare the data



Q2

Understand OLS



Q3

Compare and select
models

Q1: Data preparation

- train/test split
- missing values
- standardization
- one-hot encoding
- aligned features

Q2: OLS

- sklearn regression
- matrix formulation
- numerical diagnostics
- standard errors
- statsmodels comparison

Q3: Model selection

- full feature set
- regularization
- Pipeline + CV
- hyperparameter tuning
- final comparison

The prediction problem

Data

- A housing dataset with numerical and categorical predictors.
- Target: SalePrice.
- The first column is an identifier, not an explanatory feature.

Your task

- Build increasingly sophisticated linear models.
- Evaluate them both **in sample** and **out of sample**.
- Use cross-validation to choose hyperparameters.
- Interpret what the models and diagnostics tell you.

Keep the final test set special

It is for the final out-of-sample evaluation, not for choosing model settings.

Q1: Build a clean modelling dataset

- 1 Split the data into training and test sets.
- 2 Learn preprocessing quantities from the **training set**.
- 3 Impute missing numerical and categorical values appropriately.
- 4 Standardize numerical features using training mean and standard deviation.
- 5 One-hot encode categorical features.
- 6 Ensure the encoded test set has the **same number of columns** and the **same feature order** as the training set.

Why spend time on preprocessing?

A model can be mathematically correct but still be evaluated incorrectly if the data workflow is inconsistent.

Q2: One model, three useful viewpoints

scikit-learn

Prediction-oriented estimator
API:

```
model.fit(X, y)  
y_hat = model.predict(X)
```

Matrix algebra

Understand what OLS is
solving:

$$\hat{\beta} = \arg \min_{\beta} \|y - A\beta\|_2^2.$$

Then inspect rank, singular
values, and pseudoinverse
behaviour.

statsmodels

Inference-oriented workflow:

```
model = sm.OLS(y, A)  
result = model.fit()  
result.summary()
```

Coefficient uncertainty and
statistical diagnostics are
central.

Prediction and statistical inference have different emphases

	Machine-learning prediction	Statistical inference
Main question	How well does the fitted procedure predict unseen observations?	What can we infer about coefficients, uncertainty, and relationships?
Typical tools	train/test evaluation, cross-validation, hyperparameter selection	standard errors, confidence intervals, hypothesis tests
Useful library	scikit-learn	statsmodels
emphasis		
Project 1	central in model comparison and Q3	important in understanding OLS and Q2

Important

These are not mutually exclusive categories. The distinction is about **focus and workflow**.

The scikit-learn pattern worth learning

Most estimators share a common interface:

```
model = SomeEstimator(...)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

The same idea still works when the workflow becomes more complicated:

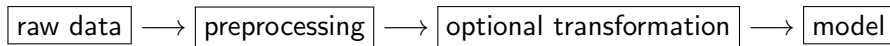
```
pipeline = Pipeline([
    ("preprocess", preprocessor),
    ("model", Ridge())
])
```

```
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
```

Why this matters

Once you understand the interface, you can change the estimator or preprocessing steps without redesigning the entire workflow.

What does a Pipeline buy you?



- One object for the complete modelling procedure.
- The same transformations are applied consistently at fit and prediction time.
- Parameters can be exposed systematically for model selection.
- Works naturally with cross-validation.
- Reduces repetitive code.
- Makes the intended modelling workflow easier to review and reuse.

Project 1

For Q3, think of preprocessing as part of the model-selection procedure, not as unrelated code that happens beforehand.

ColumnTransformer + Pipeline + GridSearchCV

1. Build the modelling pipeline

```
preprocessor = ColumnTransformer([
    ("num", numeric_pipeline,
      numeric_features),
    ("cat", categorical_pipeline,
      categorical_features)
])

pipeline = Pipeline([
    ("preprocess", preprocessor),
    ("model", SomeEstimator())
])
```

2. Tune it with cross-validation

```
search = GridSearchCV(
    pipeline,
    param_grid={...},
    cv=...,
    scoring=
        "neg_mean_squared_error",
    refit=True
)

search.fit(X_train, y_train)
```

Nested parameter names

A name such as `model__alpha` means: “set parameter alpha inside the Pipeline step named `model`.”

Cross-validation: what GridSearchCV is doing

For each candidate hyperparameter configuration:

- ① Split the training data into CV training and validation folds.
- ② Fit the complete Pipeline on the CV training folds.
- ③ Evaluate on the corresponding validation fold.
- ④ Repeat across all folds.
- ⑤ Average the validation scores and rank the candidates.

In this project

- use the same 8 folds across models;
- compare mean CV MSE and its variability;
- inspect whether the best value is at a grid boundary.

`refit=True`

After choosing the best hyperparameters, sklearn fits that best configuration once more on the **full training set**, ready for final test evaluation.

Q3: Four ways to control an unstable or overly flexible linear model

Method	Main idea	What to look at
Ridge	Shrink coefficients using an L_2 penalty.	Prediction performance and amount of shrinkage.
Lasso	Use an L_1 penalty; some feature coefficients can become exactly zero.	Prediction performance and sparsity.
Elastic Net	Combine L_1 and L_2 regularization.	Both regularization strength and mixing parameter.
Truncated PI / SVD	Remove weak singular directions before fitting the regression.	Number of retained singular directions/components.

Do not confuse these ideas

Lasso sparsifies **feature coefficients**; truncated SVD removes **directions in the design space**.

The final Project Report

Keep it concise. The report should synthesize the work rather than repeat every notebook cell.

- ① **Approach**
- ② **Model selection**
- ③ **Key results**
- ④ **Interpretation**
- ⑤ **Robustness**
- ⑥ **Limitations**

Assessment

Project grading is split between the technical implementation/results and the quality of the final report. Clear, reproducible evidence matters.

A useful rule

Outsource coding, not understanding and judgement.

A coding agent may produce code that runs, but you should still be able to judge:

- whether the train/test and CV logic is valid;
- whether the evaluation metric is interpreted correctly;
- whether a hyperparameter grid is sensible;
- whether repeated code should be refactored into a reusable pattern;
- whether Pipeline is a better abstraction than a verbose manual workflow.

The transferable skill is not memorizing API calls. It is recognizing a good modelling workflow.

Before you start

- 1 Read the full project description once before coding.
- 2 Keep the original train/test split available for later questions.
- 3 Work question by question and inspect intermediate outputs.
- 4 When tuning, keep a concise record of grids and selected values.
- 5 Do not use the held-out test set to make model-selection decisions.
- 6 Make your notebook understandable to someone who did not write it.